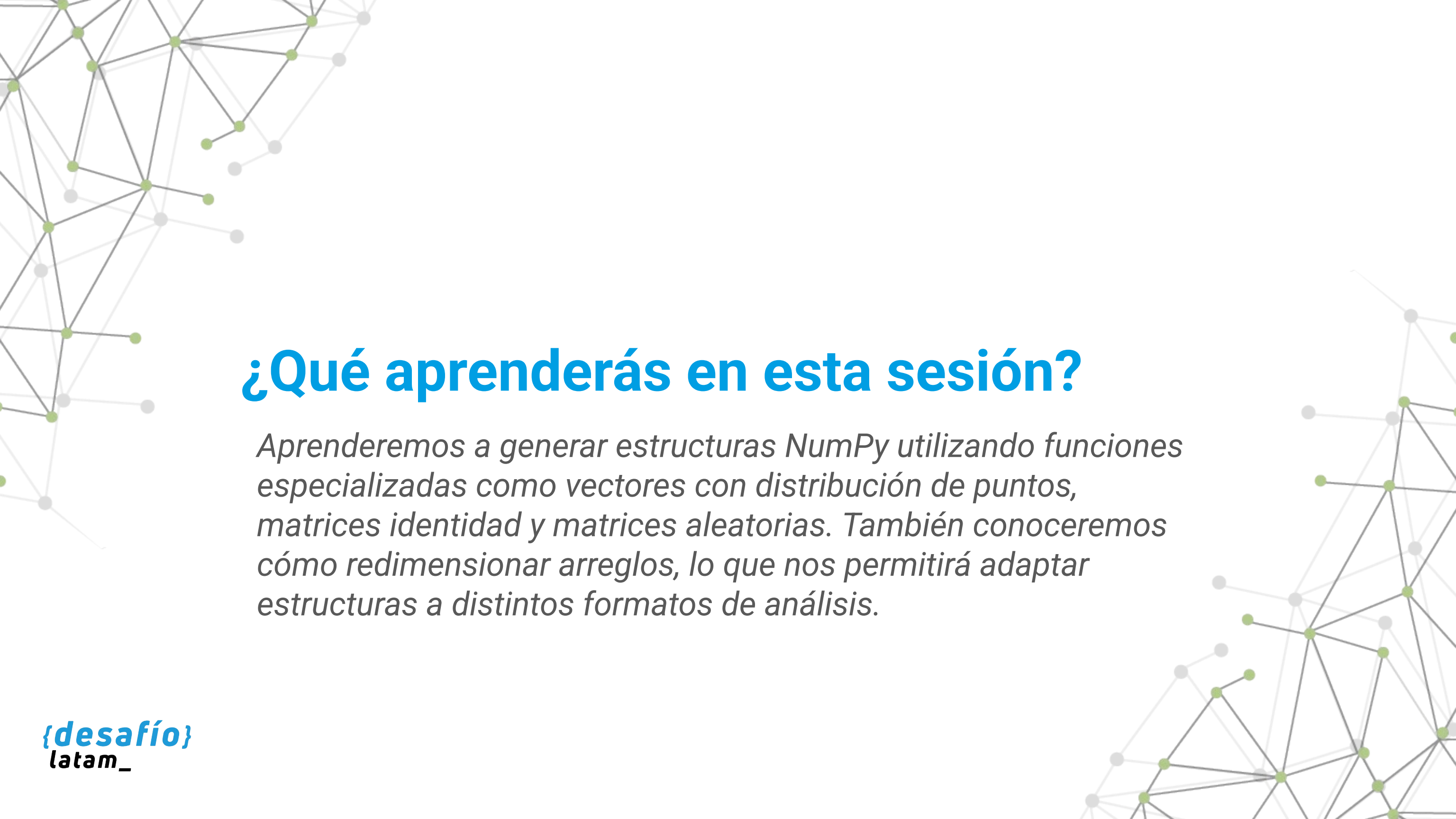




Obtención de datos y fundamentos de Numpy

**Clase 4 - Construcción de estructuras NumPy para
simulación y análisis**





¿Qué aprenderás en esta sesión?

Aprenderemos a generar estructuras NumPy utilizando funciones especializadas como vectores con distribución de puntos, matrices identidad y matrices aleatorias. También conoceremos cómo redimensionar arreglos, lo que nos permitirá adaptar estructuras a distintos formatos de análisis.

Desafío inicial

Formas parte de un equipo de simulación computacional que debe generar datos artificiales para probar un nuevo modelo de predicción de demanda. Necesitas crear arreglos con puntos equiespaciados, estructuras con propiedades especiales (como matrices identidad) y generar matrices con datos aleatorios que imiten el comportamiento real de sistemas variables.

Además, debes asegurarte de que las dimensiones de los arreglos se ajusten correctamente a los formatos que espera el modelo.

/* Vector con distribución de puntos */

¿Cómo generar estructuras simuladas útiles?

¿Cómo podrías generar estructuras simuladas que sean útiles, controladas y compatibles con un modelo de análisis numérico?

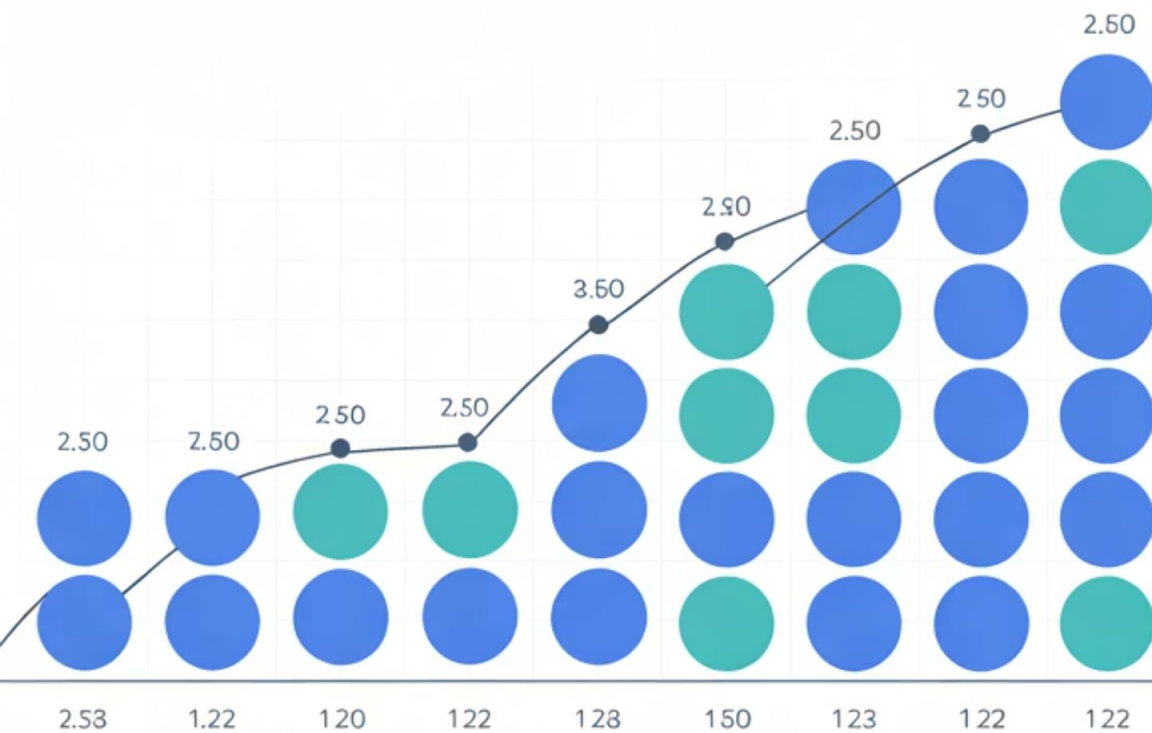
Para resolver esta necesidad, deberás conocer funciones como `linspace()`, `identity()`, `random.rand()` y `reshape()`, que permiten construir y transformar arreglos NumPy para usos específicos en ciencia de datos.



Generación de vectores con distribución de puntos

¿Qué diferencia hay entre generar puntos con `linspace()` y escribir manualmente una lista de valores?

La función `np.linspace()` permite crear un vector de valores numéricos igualmente espaciados entre dos extremos, ideal para representar escalas, tiempo, intervalos de medición o datos normalizados. A diferencia de escribir listas manuales o usar `np.arange()`, esta función garantiza que se generen exactamente `n` puntos entre los extremos especificados, incluyendo ambos límites.



¿Por qué usar linspace()?

La función `np.linspace()` permite crear un vector de valores numéricos igualmente espaciados entre dos extremos. A diferencia de `np.arange()`, garantiza que se generen exactamente `n` puntos, incluyendo los extremos.

Característica	<code>np.linspace()</code>	<code>np.arange()</code>
Inclusión del límite final	Sí, incluye el valor final	No, excluye el valor final
Precisión de espaciado	Preciso (define cantidad de puntos)	A veces impreciso por redondeo
Parámetro principal	Número de puntos	Paso o incremento

Ejemplo de linspace()

A continuación se presenta un ejemplo de código en el que se genera un vector de 5 valores equiespaciados entre 0 y 1:

```
import numpy as np
vector = np.linspace(0, 1, 5)
print(vector) # [0. 0.25 0.5 0.75 1.]
```

Este fragmento de código genera un vector de 5 valores distribuidos de forma equidistante entre 0 y 1. Es especialmente útil cuando se requiere un número exacto de puntos dentro de un intervalo definido.



Buena práctica

Usa `linspace()` cuando te interese controlar la cantidad exacta de divisiones entre dos puntos, especialmente en tareas de visualización o discretización de intervalos.

`/* Matriz identidad */`

Creación de matrices identidad

¿Por qué una matriz con 1 en la diagonal y 0 en el resto es clave en álgebra lineal?

La matriz identidad actúa como el "uno" en la multiplicación matricial: no altera el resultado al multiplicar otra matriz compatible. Por eso, es especialmente útil para simular comportamientos neutros en operaciones, validar algoritmos o inicializar estructuras en aprendizaje automático.



Identity Matrix Multiplication example

1	2	2	2	0	1
2	1	2	2	1	1
1	1	3	0	0	1
1	2	0	1	1	0
0	0	Z	Z	0	Z
0	0	Z	X	Z	X

Matrices identidad en NumPy

La función `np.identity(n)` crea una matriz identidad de tamaño $n \times n$. Este tipo de matriz es el equivalente al número 1 en la multiplicación: no altera el resultado al multiplicar otra matriz compatible. Es ampliamente usada en operaciones matriciales, validación de modelos y desarrollo de algoritmos de aprendizaje automático.

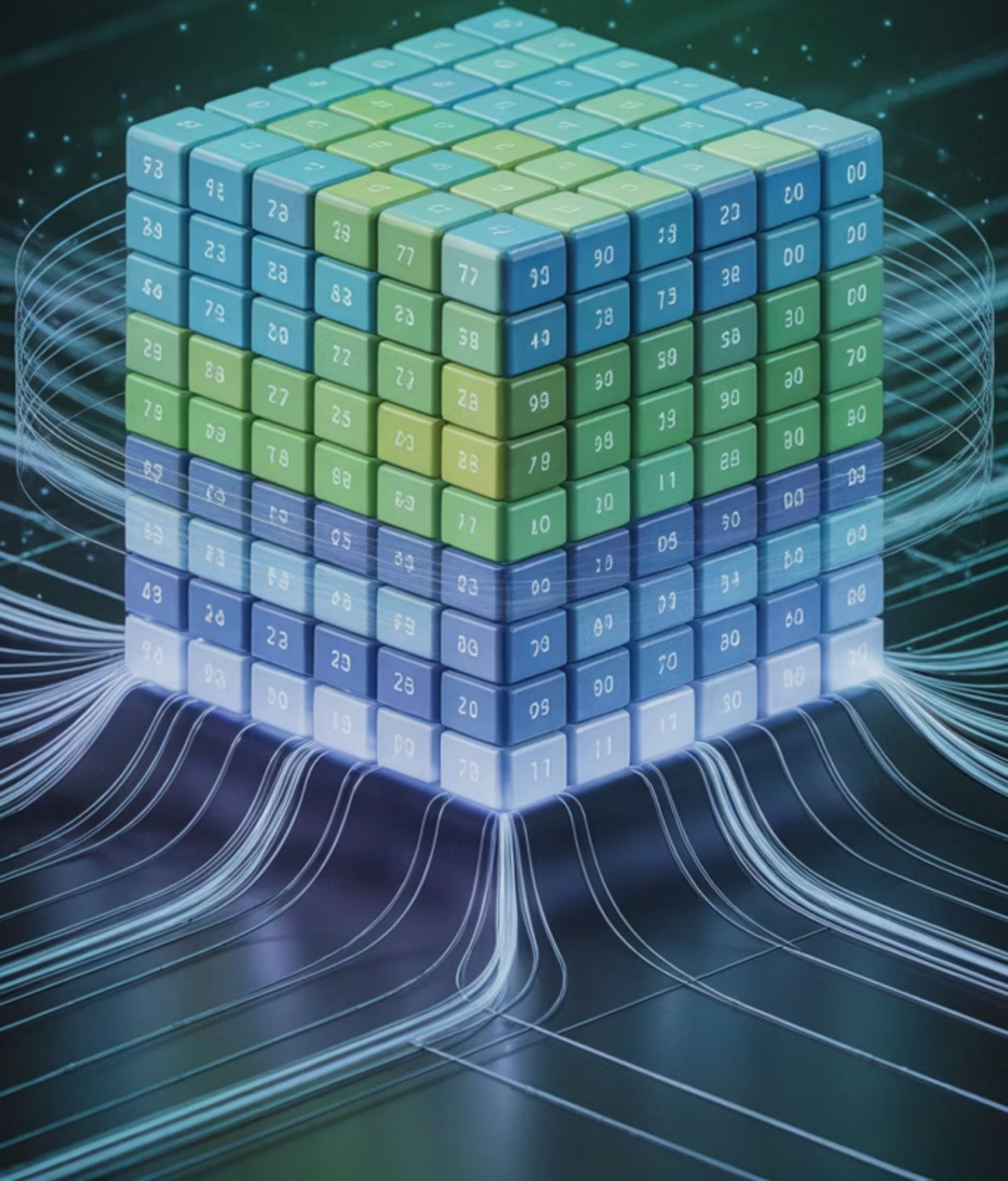
Ejemplo de matriz identidad

A continuación, se presenta un ejemplo de código para generar una matriz identidad:

```
identidad = np.identity(3)  
print(identidad)
```

Esta matriz tiene unos en la diagonal principal y ceros en el resto de los elementos.

Machine Learning Weight Initialization with Identity Matrix.



Buena práctica

Las matrices identidad son fundamentales para inicializar pesos en aprendizaje automático o verificar propiedades de algoritmos matriciales. Su comprensión es clave para quienes trabajan en análisis numérico.

/* Matriz aleatoria */

Generación de matrices aleatorias

¿Cómo puedes simular condiciones variables o generar datos de prueba sin un conjunto real disponible?

NumPy permite generar datos aleatorios controlados, ideales para simulaciones, pruebas de algoritmos o visualizaciones. Las funciones más utilizadas son:

Función	Tipo de datos	Rango de valores	Dimensiones
<code>np.random.rand(f, c)</code>	Float (decimales)	Entre 0 y 1	Definidas por el usuario
<code>np.random.randint(a, b, size)</code>	Enteros	Entre a y b-1	Definidas por el usuario

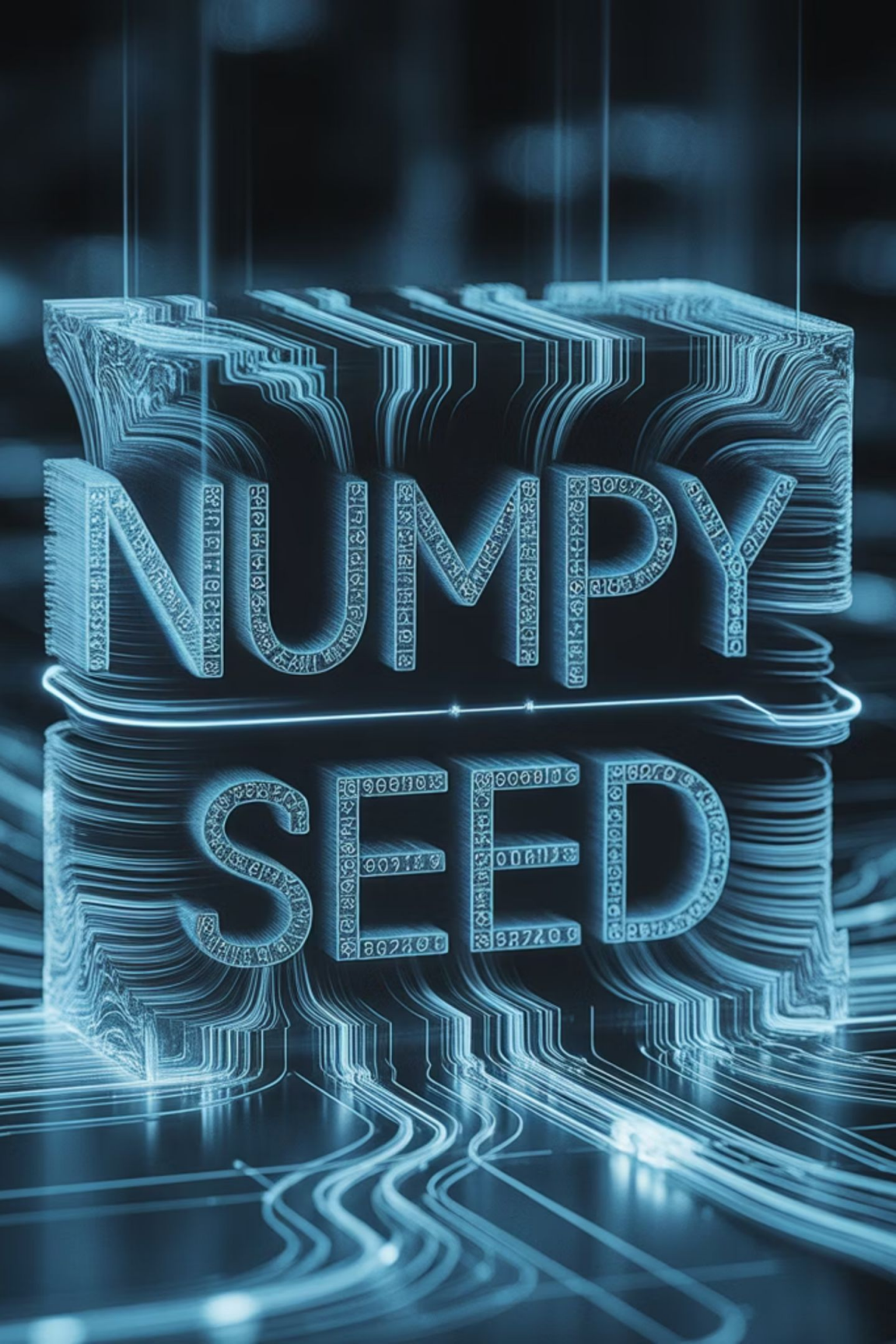
Ejemplo de matrices aleatorias

```
aleatoria = np.random.rand(2, 3)
print(aleatoria)
enteros = np.random.randint(10, 100, size=(2, 3))
print(enteros) # También puedes establecer una semilla para repetir los mismos resultados:
np.random.seed(42)
```

Buena práctica

Establecer una semilla (`np.random.seed()`) es fundamental cuando se desea reproducir resultados experimentales o análisis comparativos.

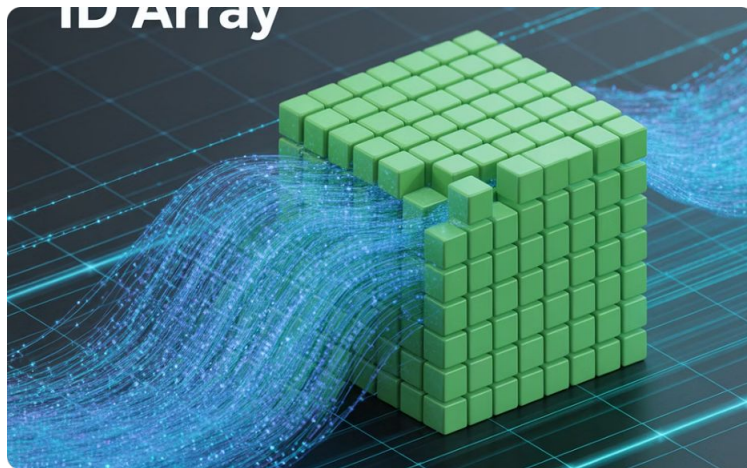
Al generar datos de forma aleatoria, es posible que necesitemos adaptarlos a las formas requeridas por nuestros modelos o herramientas.



/* Redimensionado de arreglos */

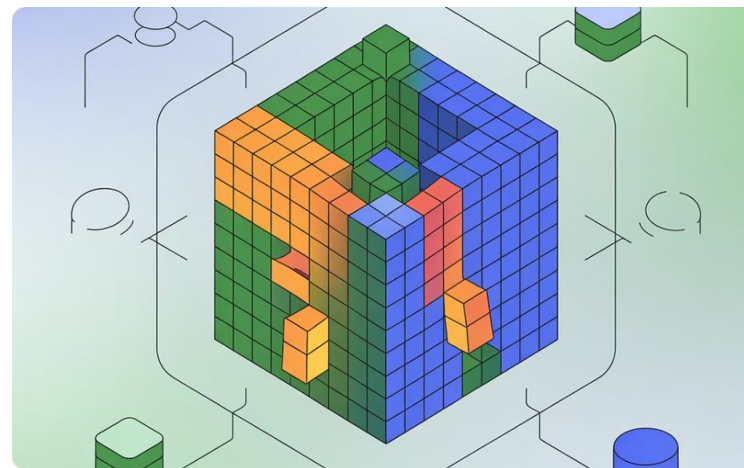
Redimensionado de arreglos

La función `reshape()` permite transformar la forma de un arreglo manteniendo la secuencia de datos. Es clave en tareas de preprocesamiento de datos, entrenamiento de modelos y manejo de estructuras multidimensionales.



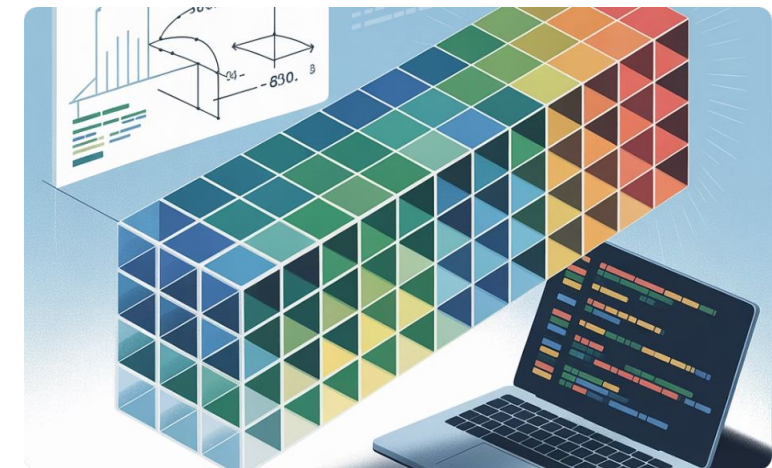
Transformación dimensional

`Reshape()` permite convertir arreglos unidimensionales en matrices bidimensionales, facilitando operaciones matriciales en algoritmos de machine learning.



Preprocesamiento de datos

Durante el preprocesamiento, `reshape()` adapta la estructura de los datos al formato requerido por modelos de aprendizaje automático sin alterar los valores originales.



Estructuras multidimensionales

El manejo eficiente de tensores y estructuras multidimensionales se logra mediante `reshape()`, conservando siempre la secuencia y cantidad total de elementos.

Ejemplo de reshape()

A continuación se presenta un ejemplo de cómo transformar un vector en una matriz de 3 filas y 4 columnas:

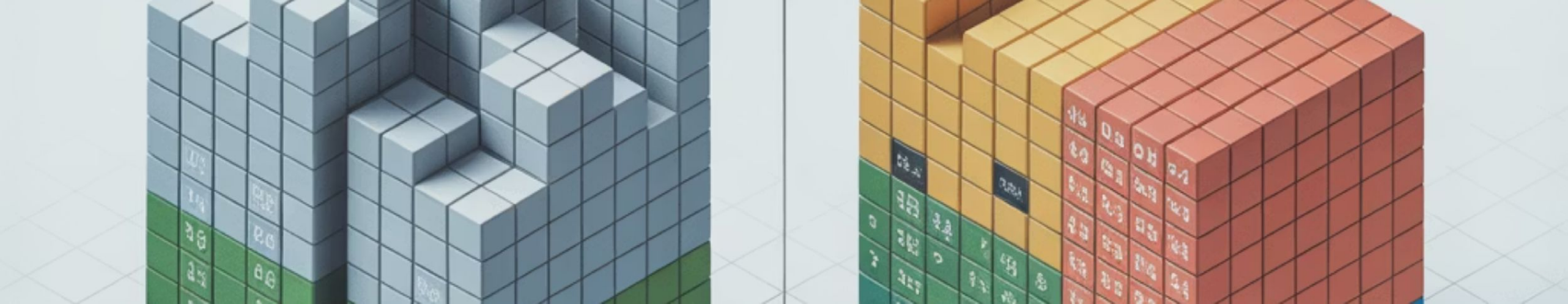
```
original = np.arange(12)
matriz = original.reshape((3, 4))
print(matriz)
```

Este fragmento de código toma un arreglo de 12 elementos secuenciales y lo reorganiza en una matriz 3x4.



Importante sobre reshape()

Importante: El producto de las dimensiones debe coincidir con la cantidad de elementos del arreglo original. Si original tiene 12 elementos, reshape() solo funcionará con combinaciones como (3,4), (2,6), (6,2), entre otros.



Funciones complementarias



ravel() o flatten(): convierten un arreglo a una sola dimensión sin alterar el original.



resize(): cambia el tamaño, pero puede alterar datos y el arreglo original.



Recomendación

Evita usar `resize()` en operaciones críticas. Prefiere `reshape()` junto con copias controladas para evitar errores difíciles de detectar.



Buena práctica

Antes de redimensionar un arreglo, asegúrate de conocer bien su forma con `.shape` y su tipo de dato con `.dtype` para evitar transformaciones incorrectas.

Actividad guiada - Simulación de sensores ambientales

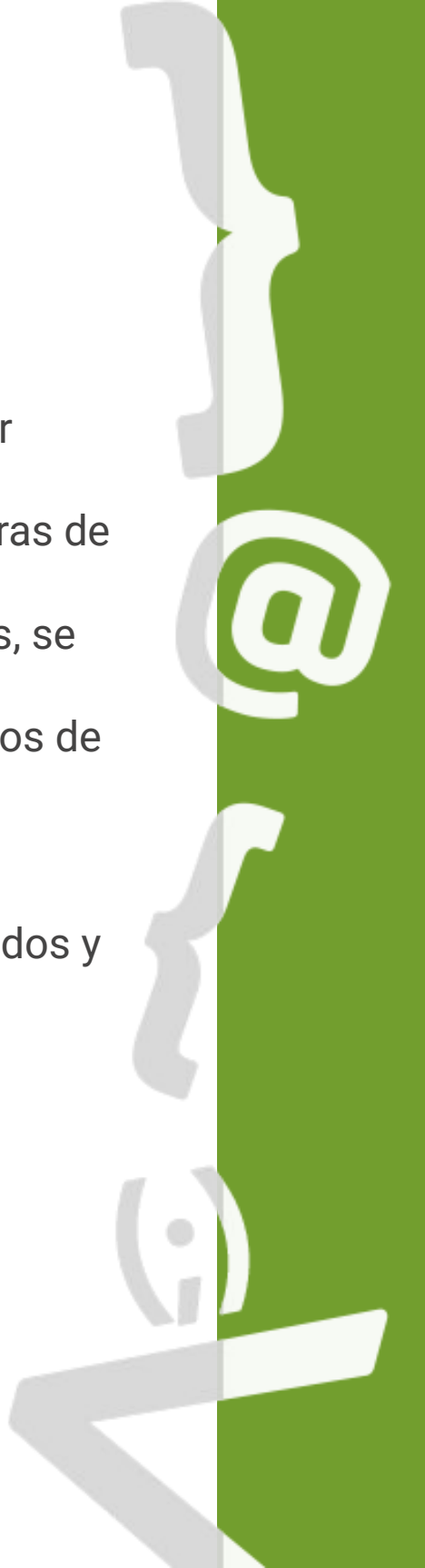


Ejercicio

Simulación de sensores ambientales

Trabajas en un laboratorio de desarrollo de soluciones para monitoreo ambiental automatizado. Tu equipo necesita probar algoritmos de visualización y control antes de que los sensores físicos estén disponibles. Para eso, deberás simular lecturas de temperatura, humedad y luminosidad en distintas condiciones (rango fijo, aleatoriedad, calibración de referencia). Además, se requiere preparar estas estructuras para su análisis posterior, utilizando transformaciones que las dejen listas para modelos de predicción.

¿Cómo podrías representar y transformar esta información usando NumPy para obtener datos de prueba confiables, variados y bien estructurados?



Ejercicio

Instrucciones

1. Lee el enunciado y analiza qué tipo de estructuras serían útiles para representar las lecturas simuladas.
2. Copia los bloques de código en tu entorno local (Jupyter Notebook, Google Colab o VSCode).
3. Ejecuta y observa el comportamiento de cada bloque.
4. Comenta en el código (usando #) qué función cumple cada línea.
5. Prueba cambiar parámetros (tamaño, valores, distribución) para simular otros escenarios.

```
import numpy as np# 1. Simular 10 lecturas de temperatura entre 15°C y 30°C con linspace
temperaturas = np.linspace(15, 30, 10)
print("Lecturas simuladas de temperatura (°C):\n", temperaturas)

# 2. Crear una matriz identidad para validar referencia (ideal de calibración)
calibracion = np.identity(3)
print("\nMatriz identidad de referencia para sensores:\n", calibracion)
```


Ejercicio

Instrucciones

3. Generar 5 lecturas aleatorias de humedad entre 40% y 80%

```
humedad = np.random.randint(40, 81, size=5)
print("\nLecturas aleatorias de humedad (%):\n", humedad)
```

4. Simular 3 series de luminosidad entre 200 y 800 lúmenes

```
luminosidad = np.random.randint(200, 801, size=(3, 5))
print("\nLecturas de luminosidad en 3 sensores (lúmenes):\n", luminosidad)
```

5. Reestructurar datos de luminosidad para análisis por minuto

```
luminosidad_lineal = luminosidad.flatten()
print("\nLecturas de luminosidad aplanadas (por minuto):\n", luminosidad_lineal)
```

6. Preparar arreglo para insertar nuevas mediciones

```
estructura = np.zeros((3, 5))
estructura[:, :] = np.random.randint(100, 500, size=(3, 5))
print("\nEstructura lista para nuevas mediciones (datos aleatorios insertados):\n", estructura)
```

Ejercicio

Preguntas de reflexión

- ☐ ¿Qué ventajas ofrece linspace() frente a escribir los valores uno por uno?
- ☐ ¿Para qué casos reales podría utilizarse una matriz identidad en la simulación de sensores?
- ☐ ¿Qué situaciones justifican el uso de randint() para representar datos ambientales?
- ☐ ¿En qué escenarios necesitas usar reshape() o flatten() antes de aplicar modelos?
- ☐ ¿Cómo se podrían combinar estos arreglos para construir un dataset completo?

Resumen

 Aprendimos a utilizar `np.linspace()` para generar secuencias numéricas con cantidad de puntos definida, útil en simulaciones y modelado de fenómenos continuos.

 Conocimos la función `np.identity()` y su utilidad en contextos de álgebra lineal y calibraciones en ciencia de datos.

 Usamos funciones del submódulo `np.random` para generar datos aleatorios realistas y reproducibles mediante el uso de semillas (seed).

 Aplicamos `reshape()` para cambiar la forma de arreglos sin alterar sus datos, permitiendo adaptar estructuras a los requerimientos del análisis.

 Utilizamos `flatten()` para transformar arreglos multidimensionales en vectores, paso previo habitual en técnicas de preprocesamiento de datos.



Próxima sesión...

En la siguiente sesión exploraremos operaciones fundamentales sobre arreglos NumPy. Aprenderemos a seleccionar elementos, aplicar condiciones, diferenciar entre referencias y copias, realizar operaciones entre arreglos o con escalares, y utilizar funciones matemáticas integradas. Estas herramientas nos permitirán transformar estructuras de datos en formas útiles para el análisis y la toma de decisiones.